Experiment 1: Student Marks, Grace Marks and Result

Aim

To calculate the total marks and percentage of a student in three subjects, add grace marks to scores from 30 to 34, and display PASS or FAIL.

Procedure / Algorithm

1. Start the program and create an empty list to store the three subject marks.
2. Read marks for three subjects one by one.
3. For each mark, if it is at least 30 but less than 35, change it to 35 and display the grace-marks message.
4. Append each adjusted mark to the list.
5. Calculate the total using the sum of the list and calculate percentage by dividing the total by 3.
6. If the minimum adjusted mark is at least 35, display PASS; otherwise, display FAIL.

Flowchart

Start → Read 3 subject marks → Check each mark (30 ≤ mark < 35)? → If yes, set it to 35 → Store marks → Calculate total and percentage → Check minimum mark ≥ 35? → Display PASS / FAIL → End

Sample Output

```
Enter marks: 32
Grace marks added
Enter marks: 65
Enter marks: 70
Total Marks: 170
Percentage: 56.6666666667
Result: PASS
```

Program Code (as supplied)

```
marks = []

for i in range(3):
    m = int(input("Enter marks: "))

    if m < 35 and m >= 30:
        m = 35
        print("Grace marks added")

    marks.append(m)

total = sum(marks)
percentage = total / 3

print("Total Marks:", total)
print("Percentage:", percentage)

if min(marks) >= 35:
    print("Result: PASS")
else:
    print("Result: FAIL")
```

Note

The percentage is calculated from the adjusted marks, because the program updates a mark before storing it.

Experiment 2: Scholarship Eligibility

Aim

To check whether a student is eligible for a scholarship using aggregate marks, subject marks, attendance and family income.

Procedure / Algorithm

7. Start the program.
8. Read the student's name, aggregate marks, subject marks, attendance and family income.
9. Check whether aggregate marks are at least 75, subject marks are at least 35, attendance is at least 75, and family income is at most Rs. 2,00,000.
10. If all four conditions are true, display Scholarship Eligible.
11. Otherwise, display Scholarship Not Eligible.

Flowchart

```
Start → Read student details and eligibility values → Check all four conditions → All satisfied? →
Yes: Scholarship Eligible / No: Scholarship Not Eligible → End
```

Sample Output

```
Enter student name: Ravi
Enter aggregate marks: 80
Enter subject marks: 40
Enter attendance: 85
Enter family income: 150000
Scholarship Eligible
```

Program Code (as supplied)

```
name = input("Enter student name: ")
marks = int(input("Enter aggregate marks: "))
subject = int(input("Enter subject marks: "))
attendance = int(input("Enter attendance: "))
income = int(input("Enter family income: "))

if marks >= 75 and subject >= 35 and attendance >= 75 and income <= 200000:
    print("Scholarship Eligible")
else:
    print("Scholarship Not Eligible")
```

Note

All four conditions must be true because the program uses the AND operator.

Experiment 3: Electricity Bill Calculation

Aim

To calculate an electricity bill using slab-wise unit charges, add a fixed charge, and apply a surcharge or rebate based on the bill amount.

Procedure / Algorithm

12. Start the program and read the number of units consumed.
13. If units are 100 or less, charge Rs. 2 per unit.
14. If units are above 100 and up to 200, charge Rs. 2 for the first 100 units and Rs. 3 for each additional unit.
15. If units are above 200, charge Rs. 2 for the first 100 units, Rs. 3 for the next 100 units and Rs. 5 for each unit above 200.
16. Add the fixed charge of Rs. 50.
17. If the bill is greater than Rs. 1,000, add a 5% surcharge; otherwise subtract a rebate of Rs. 20.
18. Display the final electricity bill.

Flowchart

Start → Read units → Select applicable unit slab → Calculate energy charge → Add Rs. 50 fixed charge → Bill > Rs. 1,000? → Yes: add 5% / No: subtract Rs. 20 → Display bill → End

Sample Output

```
Enter units consumed: 150
Electricity Bill: Rs. 420
```

Program Code (as supplied)

```
units = int(input("Enter units consumed: "))

if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5

bill = bill + 50

if bill > 1000:
    bill = bill * 1.05  # 5% surcharge
else:
    bill = bill - 20  # Rebate

print("Electricity Bill: Rs.", bill)
```

Note

For 150 units: 100×2 + 50×3 = Rs. 350; fixed charge Rs. 50; rebate Rs. 20; final bill Rs. 380. The sample output below is therefore Rs. 380.

Experiment 4: Hospital Bill and Payment Category

Aim

To calculate a hospital bill from consultation, diagnostic and treatment charges, apply a discount for larger bills, and classify the final payment.

Procedure / Algorithm

19. Start the program.
20. Read consultation, diagnostic and treatment charges.
21. Add the three charges to obtain the total.
22. If the total is zero or negative, display Invalid Bill.
23. Otherwise, if the total is at least Rs. 5,000, subtract Rs. 500.
24. Display the final bill.
25. If the final bill is at least Rs. 3,000, display High Payment; otherwise display Low Payment.

Flowchart

Start → Read 3 charges → Calculate total → Total ≤ 0? → Yes: Invalid Bill → No: Is total ≥ Rs. 5,000? → If yes subtract Rs. 500 → Display final bill → Final bill ≥ Rs. 3,000? → High / Low Payment → End

Sample Output

```
Consultation: 1000
Diagnostic: 2000
Treatment: 2500
Final Bill: 5000
High Payment
```

Program Code (as supplied)

```
c = int(input("Consultation: "))
d = int(input("Diagnostic: "))
t = int(input("Treatment: "))

total = c + d + t

if total <= 0:
    print("Invalid Bill")
else:
    if total >= 5000:
        total = total - 500

    print("Final Bill:", total)

    if total >= 3000:
        print("High Payment")
    else:
        print("Low Payment")
```

Note

The Rs. 500 discount is applied before the High Payment / Low Payment check.

Experiment 5: Vehicle Parking Fee

Aim

To calculate the parking fee based on the number of parking hours and vehicle type.

Procedure / Algorithm

26. Start the program.
27. Read the number of parking hours.
28. Read the vehicle type as car or bike.
29. If the vehicle type is car, calculate the fee at Rs. 30 per hour.
30. Otherwise, calculate the fee at Rs. 10 per hour.
31. Display the parking fee.

Flowchart

Start → Read parking hours and vehicle type → Vehicle is car? → Yes: hours × Rs. 30 / No: hours × Rs. 10 → Display parking fee → End

Sample Output

```
Enter parking hours: 3
Enter vehicle (car/bike): car
Parking Fee: 90
```

Program Code (as supplied)

```
h = int(input("Enter parking hours: "))
v = input("Enter vehicle (car/bike): ")

if v == "car":
    bill = h * 30
else:
    bill = h * 10

print("Parking Fee:", bill)
```

Note

As written, every vehicle value other than the exact string 'car' is charged at the bike rate.

Experiment 6: College Admission Eligibility

Aim

To determine admission eligibility using Maths marks, Science marks and the average percentage in three subjects.

Procedure / Algorithm

32. Start the program.
33. Read the student's name and marks in Maths, Science and English.
34. Calculate total marks by adding the three subject marks.
35. Calculate percentage by dividing the total by 3.
36. Check whether Maths marks are at least 40, Science marks are at least 40, and percentage is at least 60.
37. If all conditions are satisfied, display Admission Eligible, student name and percentage.
38. Otherwise, display Admission Not Eligible.

Flowchart

Start → Read name and 3 subject marks → Calculate total and percentage → Maths ≥ 40 AND Science ≥ 40 AND percentage ≥ 60? → Yes: display eligible details / No: display not eligible → End

Sample Output

```
Enter student name: Anu
Enter Maths marks: 70
Enter Science marks: 65
Enter English marks: 60
Admission Eligible
Student: Anu
Percentage: 65.0
```

Program Code (as supplied)

```
name = input("Enter student name: ")
maths = int(input("Enter Maths marks: "))
science = int(input("Enter Science marks: "))
english = int(input("Enter English marks: "))

total = maths + science + english
percentage = total / 3

if maths >= 40 and science >= 40 and percentage >= 60:
    print("Admission Eligible")
    print("Student:", name)
    print("Percentage:", percentage)
else:
    print("Admission Not Eligible")
```

Note

English has no separate minimum-mark condition; it contributes to the average percentage.

Experiment 7: Team Score and Team-Size Validation

Aim

To read details for multiple teams and display the score only when the team size is between 2 and 4 inclusive.

Procedure / Algorithm

39. Start the program and read the number of teams.
40. Repeat the following steps for each team using a for loop.
41. Read the team name, team size and score.
42. If the team size is from 2 through 4, display the team name and total score.
43. Otherwise, display Invalid team size.
44. Continue until all teams have been processed, then end.

Flowchart

Start → Read number of teams → Repeat for each team → Read name, size and score → Is 2 ≤ size ≤ 4? → Yes: display name and score / No: display Invalid team size → More teams? → Yes: repeat / No: End

Sample Output

```
Number of teams: 2
Team name: Tigers
Team size: 3
Enter score: 85
Tigers Total Score: 85
Team name: Eagles
Team size: 5
Enter score: 90
Invalid team size
```

Program Code (as supplied)

```
n = int(input("Number of teams: "))

for i in range(n):
    name = input("Team name: ")
    size = int(input("Team size: "))
    score = int(input("Enter score: "))

    if size >= 2 and size <= 4:
        print(name, "Total Score:", score)
    else:
        print("Invalid team size")
```

Note

The loop runs exactly the number of times entered for the number of teams.

Experiment 8: Exam Eligibility Check

Aim

To check exam eligibility based on fee payment, attendance and internal marks, and to display each unmet requirement.

Procedure / Algorithm

45. Start the program.
46. Read fee status (1 for paid, 0 for not paid), attendance and internal marks.
47. If fee status is 0, display Pay fees.
48. If attendance is below 75, display Low attendance.
49. If internal marks are below 40, display Low marks.
50. Check whether fee is paid, attendance is at least 75 and internal marks are at least 40.
51. If all conditions are satisfied, display Eligible for exam; otherwise display Not eligible.

Flowchart

```
Start → Read fee, attendance and internal marks → Display each failed requirement → fee = 1 AND attendance ≥ 75 AND marks ≥ 40? → Yes: Eligible for exam / No: Not eligible → End
```

Sample Output

```
Fee paid? (1=yes, 0=no): 1
Attendance: 80
Internal marks: 45
Eligible for exam
```

Program Code (as supplied)

```
fee = int(input("Fee paid? (1=yes, 0=no): "))
attendance = int(input("Attendance: "))
marks = int(input("Internal marks: "))

if fee == 0:
    print("Pay fees")
if attendance < 75:
    print("Low attendance")
if marks < 40:
    print("Low marks")

if fee == 1 and attendance >= 75 and marks >= 40:
    print("Eligible for exam")
else:
    print("Not eligible")
```

Note

The three warning checks are separate if statements, so more than one warning can be displayed.

Experiment 9: Grace Marks and Pass/Fail

Aim

To adjust marks by adding five grace marks within the specified range, cap the final marks at 100, and display PASS or FAIL.

Procedure / Algorithm

52. Start the program and read the original marks.
53. Store the entered marks in a separate variable named original.
54. If marks are at least 35 and less than 40, add 5 grace marks.
55. If the updated marks exceed 100, set marks to 100.
56. Display original marks and final marks.
57. If final marks are at least 40, display Pass; otherwise display Fail.

Flowchart

Start → Read original marks → Save original value → Is 35 ≤ marks < 40? → Yes: add 5 → Is marks > 100? → Yes: set marks to 100 → Display original and final marks → Final marks ≥ 40? → Pass / Fail → End

Sample Output

```
Enter original marks: 37
Original Marks: 37
Final Marks: 42
Pass
```

Program Code (as supplied)

```
marks = int(input("Enter original marks: "))

original = marks

if marks >= 35 and marks < 40:
    marks = marks + 5

if marks > 100:
    marks = 100

print("Original Marks:", original)
print("Final Marks:", marks)

if marks >= 40:
    print("Pass")
else:
    print("Fail")
```

Note

Grace marks are added only when the original mark is from 35 to 39 inclusive.

Experiment 10: Internship Eligibility

Aim

To determine internship eligibility based on marks, assessment score and attendance, and display the student's total score when eligible.

Procedure / Algorithm

58. Start the program.
59. Read student name, marks, assessment score and attendance.
60. Check whether marks are at least 60, assessment score is at least 50, and attendance is at least 75.
61. If all conditions are true, calculate total_score by adding marks and assessment score.
62. Display Eligible for Internship, student name and total score.
63. Otherwise, display Not Eligible for Internship.

Flowchart

```
Start → Read student details and scores → Check marks ≥ 60 AND assessment ≥ 50 AND attendance ≥ 75
→ All satisfied? → Yes: calculate and display total score / No: display Not Eligible for
Internship → End
```

Sample Output

```
Enter student name: Kiran
Enter marks: 70
Enter assessment score: 60
Enter attendance: 80
Eligible for Internship
Student Name: Kiran
Total Score: 130
```

Program Code (as supplied)

```
name = input("Enter student name: ")
marks = int(input("Enter marks: "))
assessment = int(input("Enter assessment score: "))
attendance = int(input("Enter attendance: "))

if marks >= 60 and assessment >= 50 and attendance >= 75:
    total_score = marks + assessment
    print("Eligible for Internship")
    print("Student Name:", name)
    print("Total Score:", total_score)
else:
    print("Not Eligible for Internship")
```

Note

Attendance is used as an eligibility condition but is not included in total_score.